

Yıldız Teknik Üniversitesi Bilgisayar Mühendisliği

Yazılım Mühendisliği Dersi Proje Ödevi

Dersin Yürütücüsü: Prof. Dr. Mehmet Sıddık AKTAŞ

Mayıs, 2025

Konu: Gider Yönetim Sistemi

21011079 – Cabir COŞKUN

21011065 – Erdem KARACA

21011070 – Ömer Faruk YILDIRIM

21011602 – Yusuf DAMBASAN

21011022 – Tarık TAŞKIN

İÇİNDEKİLER

1. **PROJE PLANI** 3

1.1 Proje Alan Tanımı 3

KABULLER 3

KISITLAR 4

1.2 Proje İş-Zaman Çizelgesi 5

1.3 Ekip Organizasyon Şeması ve Görev Dağılımları 5

1.4 Risk Tablosu 6

2. İSTEKLERİN MODELLENMESİ 6

2.1 Kullanım Senaryosu Modellemesi 6

2.2 İzlenebilirlik Tablosu 10

3. NESNEYE DAYALI MODELLEME 11

3.1 Sınıf Diyagramı 11

4. NESNEYE DAYALI TASARIM 11

4.1 Sıralama (Sequence) Diyagramı 11

4.2 Durum (State) Diyagramı 14

4.3 Etkinlik (Activity) Diyagramı 18

5. TESTLER 24

1. PROJE PLANI

1.1 Proje Alan Tanımı

Gider Yönetim Sistemi, şirket çalışanlarının iş amacıyla sahada gerçekleştirdiği taksi, otopark, yakıt gibi harcamaların kayıt altına alınması, yöneticiler tarafından onaylanması, muhasebe tarafından incelenip tazmin edilmesi ve tüm bu işlemlerin raporlanabilmesini sağlayan bir bilgi sistemi çözümüdür. Bu sistem sayesinde manuel harcama formları ve belge takibi gibi zaman alan süreçler dijitalleştirilerek iş verimliliği artırılacak, harcama denetimi sağlanacak ve bütçe planlaması daha sağlıklı yapılacaktır.

KABULLER

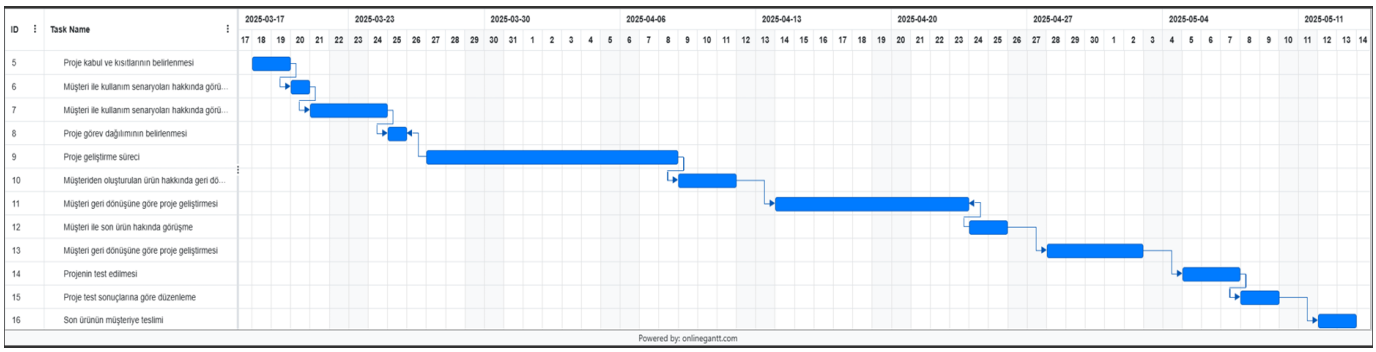
1. **Kullanıcı Rolleri ve Yetkileri**
    - Sistemde üç temel kullanıcı rolü vardır: Çalışan, Birim Yöneticisi, Muhasebe Yetkilisi. Üst yönetim işlemleri ise sistem yöneticileri tarafından temsil edilir.
    - Her çalışan yalnızca kendi harcamasını kaydedebilir.
    - Yöneticiler yalnızca kendi birimlerinde bulunan çalışanların harcamalarını onaylayabilir.
    - Muhasebe birimi, onaylı harcamaları işleyerek tazmin sürecini yürütür.
    - Üst yönetim, bütçe tanımlama ve eşik değer belirleme gibi yetkilere sahiptir.
2. **Harcama Kalemleri ve Kategorileri**
    - Harcamalar; Taksi, Otopark, Benzin vb. önceden tanımlanmış kategoriler altında yapılır.
    - Yanlış kaleme yapılan harcama talepleri muhasebe birimi tarafından uygun kaleme aktarılır.
3. **Bütçe ve Limit Yönetimi**
    - Her birim ve harcama kalemi için yıllık bütçeler üst yönetim tarafından tanımlanır.
    - Limit aşımlarında, yalnızca aşıma neden olan son harcama talebi kabul edilir (eğer aşım eşik değeri geçmiyorsa).
    - Eşik değerin geçilmesi durumunda, yalnızca eşik altı kısım tazmin edilir, kalan kısım reddedilir.

4. **Süreç Akışı**
    - Harcama girişi → Birim yöneticisi onayı → Muhasebeye belge sunumu ve tazmin süreci.
    - Süreç tamamlanmadan ödeme yapılmaz.
    - Belge sunumu fiziksel olarak yapılmakta; sistem yalnızca dijital kayıt ve izleme içindir.
5. **Raporlama ve Tahmin**
    - Sistem, kişi, birim, tarih ve kalem bazında tablo ve grafiklerle detaylı raporlar sunar.
    - Gelecek yıl için harcama tahmini, geçmiş yılların verileriyle oluşturulan Hareketli Ortalama Yöntemi ile yapılır.
    - Hareketli ortalama yöntemi en fazla 5 yıl verisi ile çalışır ve yalnızca bir sonraki yıl için öneri sunar.

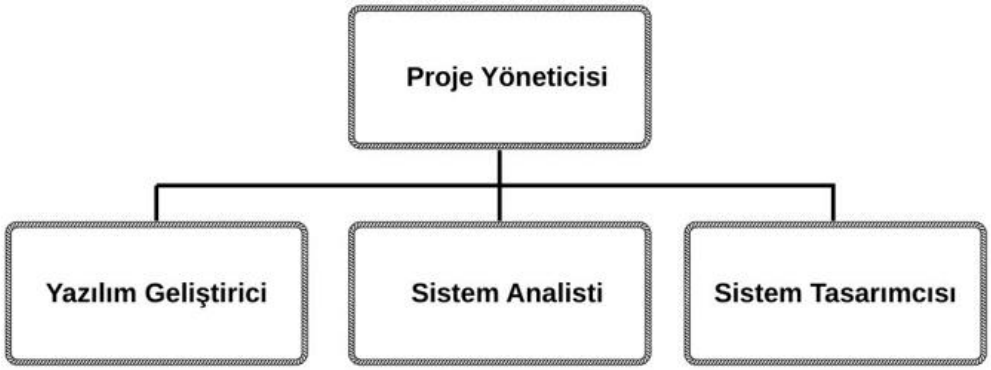
KISITLAR

1. **Sistem Dışı İşlemler**
    - Fiziksel belgelerin kontrolü ve doğrulama işlemleri sistem dışında yürütülür.
    - Sistemin belge sahteciliğini tespit edecek yeteneği yoktur.
2. **Gerçek Zamanlı Bütçe Güncellemesi**
    - Bütçeler yıllık olarak tanımlanır ve yıl içerisinde güncellenemez. (Varsayımsal kısıt)
3. **Eşik Değer**
    - Eşik değerin üzerindeki taleplerin tazmin edilmeyen kısmı geri ödenmez ve bu bilgi kullanıcıya sistem içi bildirim ile iletilir.
4. **Yetki Kısıtları**
    - Yöneticiler ve muhasebe personeli yalnızca yetkili oldukları personel ve birimler üzerinde işlem yapabilir.
5. **Tahmin Yöntemi Kısıtları**
    - Sadece basit hareketli ortalama yöntemi kullanılabilir. Regresyon, zaman serisi analizi gibi ileri düzey tahmin teknikleri kullanılmaz.

1.3 Proje İş-Zaman Çizelgesi



1.4 Ekip Organizasyon Şeması ve Görev Dağılımları



Görevler ve Görevliler

Proje Yöneticisi – Erdem KARACA

Yazılım Geliştirici – Tarık Taşkın, Ömer Faruk Yıldırım

Sistem Analisti – Erdem Karaca, Cabir Coşkun

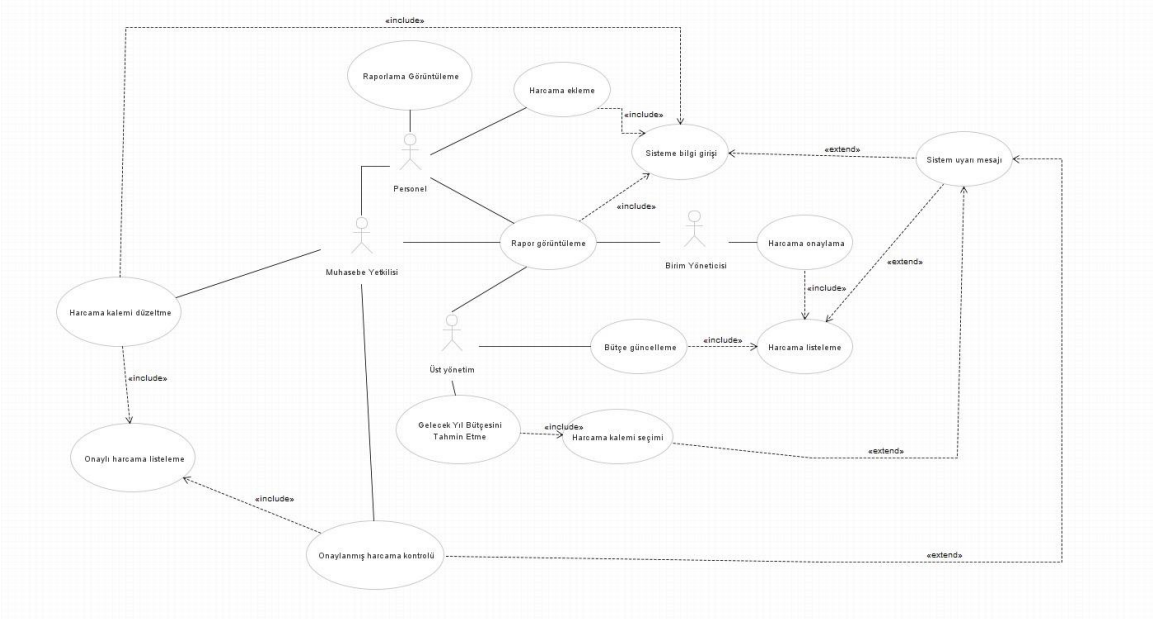
Sistem Tasarımcısı - Yusuf Dambasan

Risk Tablosu

Risk No	Risk Açıklaması	Etki	Olasılık	Önlem
1	Gereksinimlerin eksik toplanması	Gereksinimlerin eksik belirlenmesi proje boyunca hatalara ve yeniden çalışma ihtiyacına yol açabilir.	Orta	Kullanıcılarla erken ve detaylı görüşmeler yapılması.
2	Anahtar personelin projeden ayrılması	Projede bilgi eksikliği oluşabilir ve ilerleme sürecinde ciddi yavaşlamalar yaşanabilir.	Düşük	Bilgi paylaşımı kültürü oluşturulmalı ve iyi dokümantasyon sağlanmalı.
3	Zaman planına uyulamaması	Belirlenen sürelerin aşılması projede gecikmelere ve maliyet artışına neden olabilir.	Orta	Proje aşamaları sıkı takip edilmeli, düzenli ara testler yapılmalı.
4	Teknik sorunlar (entegrasyon, veri uyuşmazlığı)	Teknik aksaklıklar sistemin beklenen şekilde çalışmamasına ve kullanıcı memnuniyetsizliğine yol açabilir.	Orta	Erken aşamada teknik prototip oluşturulmalı ve test ortamı hazırlanmalı.

2. İSTEKLERİN MODELLENMESİ

2.1 Kullanım Senaryosu Modellemesi



Kullanım Senaryosu S1 – Personelin Harcama Girişi Yapması

Birincil Aktör	Personel
İlgililer / Beklentiler	Personel, iş için yaptığı harcamayı sisteme girmek ister.
Ön Koşullar	Personel sisteme giriş yapmış ve harcama formu ekranına ulaşmış olmalıdır.
Son Koşullar	Harcama kaydı başarıyla oluşturulur ve 'pending' durumu ile yöneticinin onayına düşer.
Ana Senaryo	1. Personel 'Yeni Harcama' sekmesine tıklar. 2. Harcamaya ait tutarı girer. 3. Harcamaya ait belgeyi yükler. 4. Harcama kalemini seçer (örneğin: taksi, otopark). 5. 'Başvur' butonuna tıklar.
Alternatif Akış	5.a Eksik bilgi girişi: 1. Personel gerekli alanlardan birini boş bırakır. 2. Sistem hata mesajı gösterir ve kayda izin vermez.

Kullanım Senaryosu S2 – Yöneticinin Harcamayı Onaylaması

Birincil Aktör	Birim Yöneticisi
İlgililer / Beklentiler	Yönetici, personelinin yaptığı harcamayı inceleyip onaylamak ister.
Ön Koşullar	Yönetici sisteme giriş yapmış olmalı ve bekleyen harcamalar listesine erişmelidir.

Son Koşullar	Harcama kaydı 'approved' statüsüne geçer ve muhasebenin ekranına düşer.
Ana Senaryo	1. Yönetici 'Harcama Onay' sekmesine tıklar. 2. Bekleyen (pending) durumdaki harcamaları listeler. 3. İncelediği harcama için 'Onayla' butonuna tıklar.
Alternatif Akış	2.a Harcama listelenememesi: 1. Sistem üzerinde henüz pending durumda harcama yoksa liste boş olur.

Kullanım Senaryosu S3 – Muhasebenin Harcamayı İşlemesi

Birincil Aktör	Muhasebe Yetkilisi
İlgililer / Beklentiler	Muhasebe, onaylanmış harcamayı inceler, kalemi gerekirse düzeltir, limit durumuna göre karar verir ve durumu günceller.
Ön Koşullar	Muhasebe personeli sisteme giriş yapmış ve approved durumdaki harcama listesine erişmiş olmalıdır.
Son Koşullar	Harcama kaydı işlenir ve 'paid' ya da 'rejected' durumuna geçer. Limit aşımı ve eşik değeri kontrolü yapılır.
Ana Senaryo	1. Muhasebe 'Onaylı Harcamalar' sekmesine tıklar. 2. Listeden bir harcama seçer. 3. Gerekirse harcama kalemini düzeltir. 4. Sistemde kalem bütçesi ve eşik değeri kontrol edilir. 5. Limit henüz aşılmamışsa tam ödeme yapılır. 6. Limit aşıldıysa ama eşik değeri aşılmadıysa ödeme yapılır ve kalan çalışanlara o kalem için bildirim gönderilir. 7. Eşik değer de aşılmışsa sadece eşik altı kısım ödenir, kalan kısım reddedilir. 8. Durum 'paid' ya da 'rejected' olarak güncellenir.
Alternatif Akış	3.a Kalem hatalı seçilmemişse işlem doğrudan yapılır. 8.a Durum seçilmemişse sistem uyarı verir ve işlem yapılmaz.

Kullanım Senaryosu S4 – Üst Yönetimin Bütçe Güncellemesi Yapması

Birincil Aktör	Üst Yönetim

İlgililer / Beklentiler	Üst yönetim, harcama kalemlerinin yıllık bütçesini güncellemek ister.
Ön Koşullar	Üst yönetim sisteme giriş yapmış ve bütçe yönetim ekranına ulaşmış olmalıdır.
Son Koşullar	Yeni bütçe değeri sisteme kaydedilir ve ilgili kalemler yeni limitlerle güncellenir.
Ana Senaryo	1. Üst yönetim 'Bütçe Yönetimi' sekmesine tıklar. 2. Güncellemek istediği harcama kalemini seçer. 3. Yeni bütçe miktarını girer. 4. 'Kaydet' butonuna tıklar.
Alternatif Akış	3.a Eksik bilgi girişi: 1. Üst yönetim bütçe miktarını boş bırakır. 2. Sistem hata mesajı gösterir.

Kullanım Senaryosu S5 – Üst Yönetimin Gelecek Yıl Bütçesini Tahmin Etmesi

Birincil Aktör	Üst Yönetim
İlgililer / Beklentiler	Üst yönetim, geçmiş yılların harcama verilerine göre yeni yıl için bütçe önerisi alır.
Ön Koşullar	Üst yönetim sisteme giriş yapmış ve tahmin ekranına ulaşmış olmalıdır.
Son Koşullar	Sistem Hareketli Ortalama Yöntemi ile önerilen bütçeyi gösterir.
Ana Senaryo	1. Üst yönetim 'Bütçe Tahmin' sekmesine tıklar. 2. Harcama kalemini seçer. 3. 'Tahmin Et' butonuna tıklar. 4. Sistem geçmiş 5 yıl verisine göre ortalamayı hesaplayarak tahmin sonucunu gösterir.
Alternatif Akış	2.a Yeterli veri yoksa sistem uyarı verir ve işlem iptal edilir.

Kullanım Senaryosu S6 – Raporlama Görüntüleme

Birincil Aktör	Üst Yönetim
İlgililer / Beklentiler	Üst Yönetim, harcamaları kişi, birim, kalem ve tarihe göre tablo ve grafik olarak görmek ister.
Ön Koşullar	Üst Yönetim sisteme giriş yapmış ve raporlama ekranına ulaşmış olmalıdır.
Son Koşullar	İlgili filtrelere göre tablo ve grafik gösterilir.

Ana Senaryo	1. Üst Yönetim 'Raporlama' sekmesine tıklar. 2. Kişi, birim, kalem ve tarih filtrelerinden seçim yapar. 3. 'Raporla' butonuna tıklar. 4. Sistem verileri tablo ve grafik şeklinde gösterir.
Alternatif Akış	2.a Filtre seçilmemesi durumunda sistem tüm verileri raporlar.

Kullanım Senaryosu S7 – Giriş Yapma

Birincil Aktör	Tüm Roller (Personel, Muhasebe, Yönetici, Üst Yönetim)
İlgililer / Beklentiler	Kullanıcı sisteme mail ve şifre ile giriş yapmak ister.
Ön Koşullar	Kullanıcı sistem arayüzüne erişmiş olmalıdır.
Son Koşullar	Kullanıcının rolüne uygun arayüz açılır (ör. personel ana sayfası).
Ana Senaryo	1. Kullanıcı giriş ekranına gelir. 2. Mail ve şifre bilgilerini girer. 3. 'Giriş Yap' butonuna tıklar. 4. Sistem bilgileri veritabanında kontrol eder. 5. Bilgiler doğruysa rol bilgisine göre ilgili sayfa açılır.
Alternatif Akış	2.a Eksik bilgi girişi: - Sistem uyarı mesajı gösterir (ör. 'Mail ve şifre boş bırakılamaz.'). 4.a Bilgiler veritabanında eşleşmez: - Sistem 'Kullanıcı bulunamadı' veya 'Hatalı şifre' uyarısı verir.

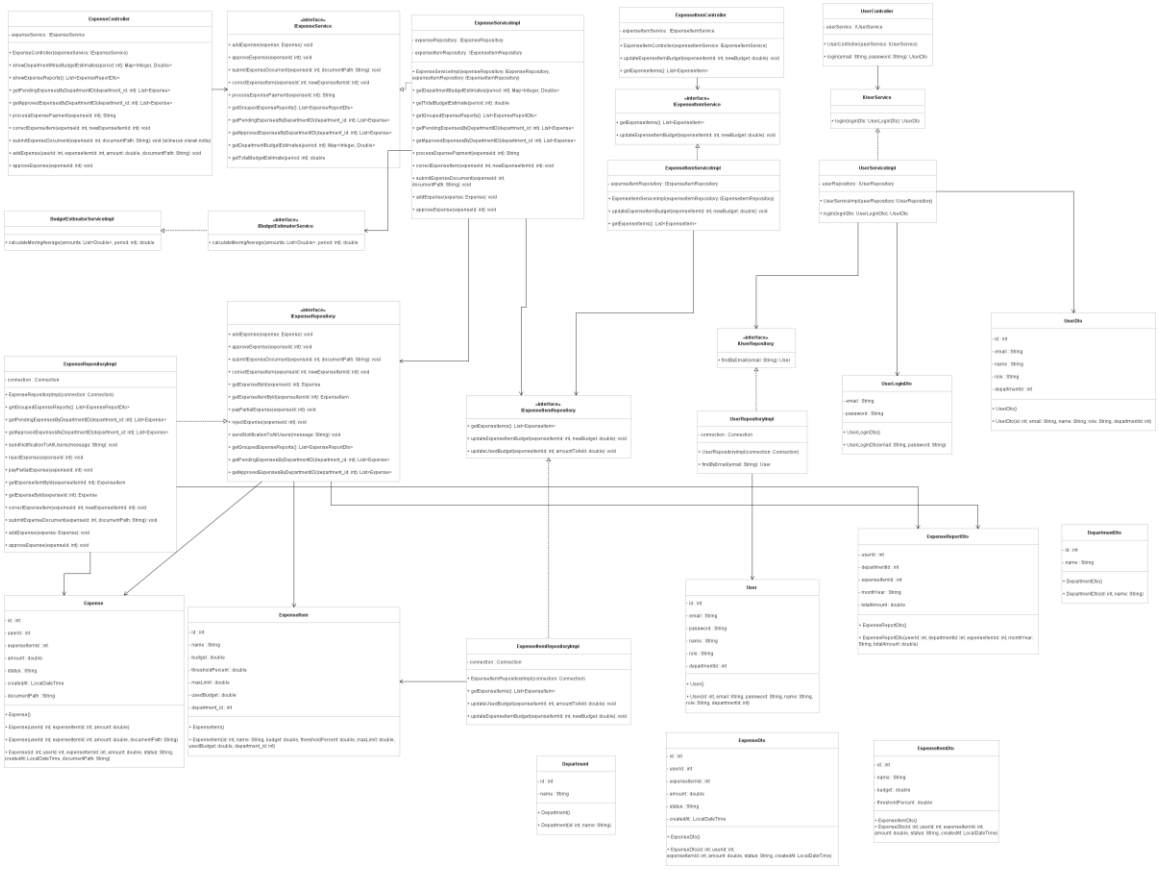
2.2 İzlenebilirlik Tablosu

Gereksinimler	Personel	Muhasebe	Yönetici	Üst Yönetim
Sisteme giriş yapabilme	X	X	X	X
Harcama kaydı ekleme	X			
Harcama belgesi ekleme	X			
Harcama onaylama			X	
Harcama kalemi düzeltme		X		
Harcama tanzim etme		X		

Departmana ve harcama kalemine göre bütçe belirleme				X
Harcama raporlarını görüntüleme				X
Gelecek için bütçe tahmini görüntüleme				X

3. NESNEYE DAYALI MODELLEME

3.1 Sınıf Diyagramı



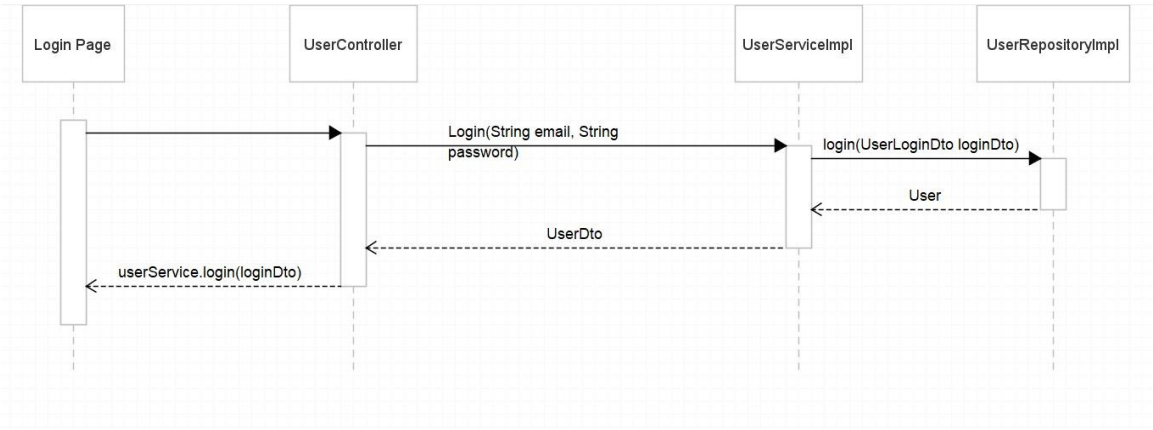
UML diyagramı burada ufak gözüktüğünden dolayı ayrıca bu linkten erişebilirsiniz:

https://drive.google.com/file/d/1OZh0siaMF3ks9pNxoJKeAqig8KGMTw0v/view?usp=sharing

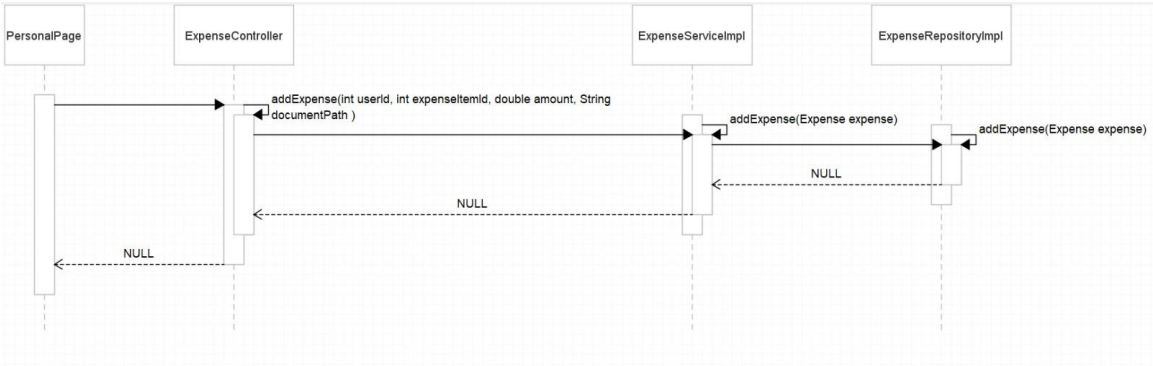
4.NESNEYE DAYALI TASARIM

4.1 Sıralama (Sequence) Diyagramları

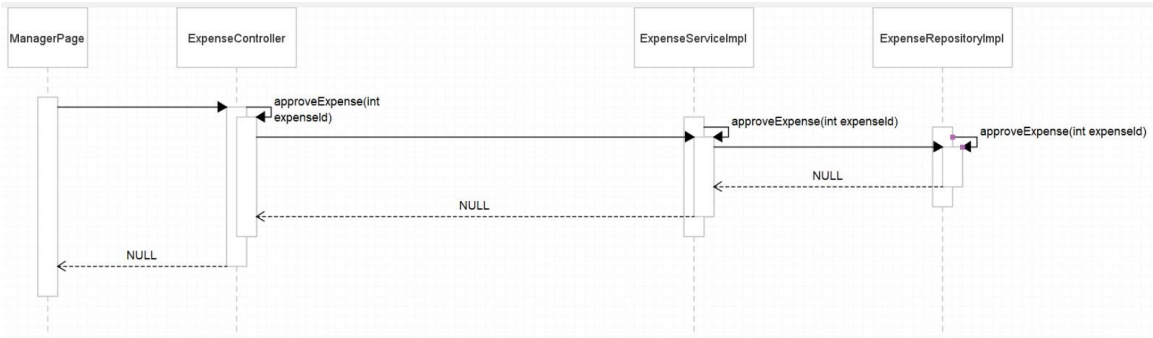
4.1.1 Sisteme Giriş Sequence Diyagramı



4.1.2 Personel Harcama Ekleme Sequence Diyagramı



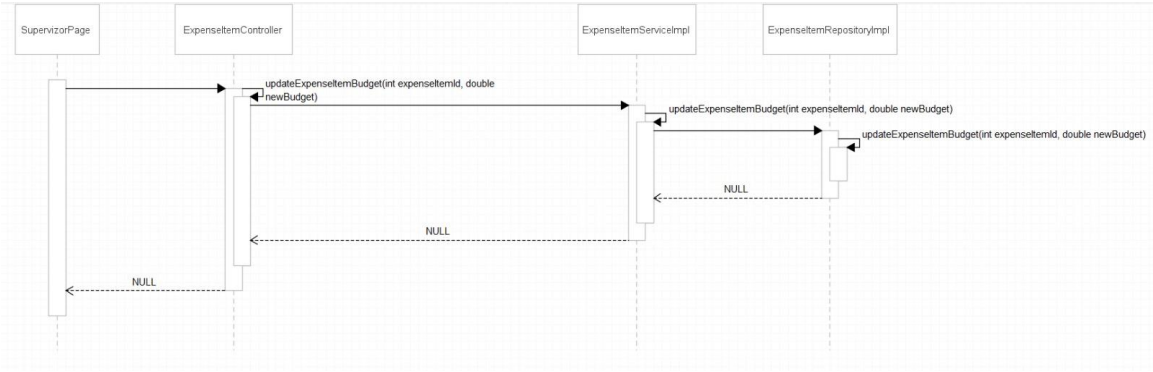
4.1.3 Birim Yöneticisi Harcama Onaylama Sequence Diyagramı



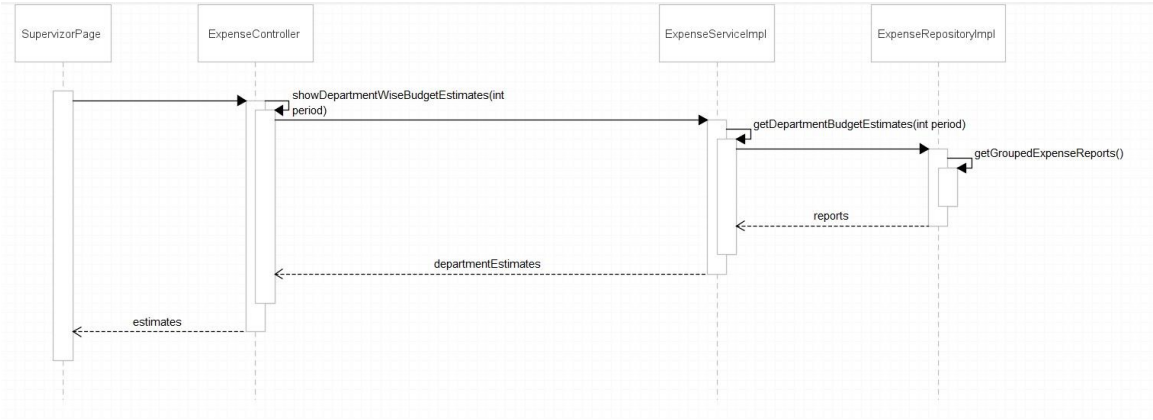
4.1.4 Muhasebe Harcama Tanzim Etme Sequence Diyagramı



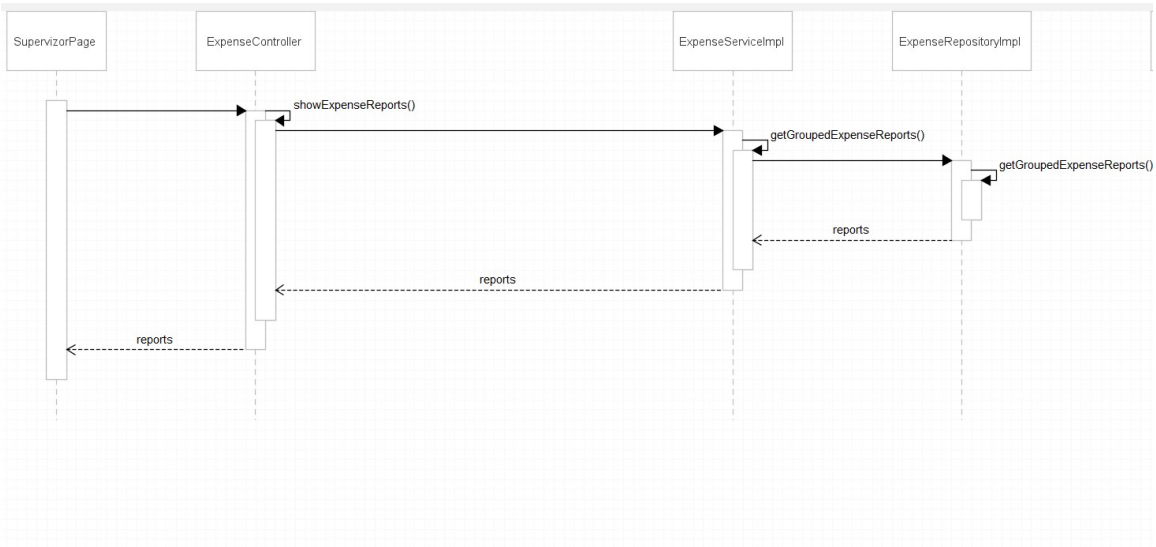
4.1.5 Üst Yönetim Bütçe Güncelleme Sequence Diyagramı



4.1.6 Üst Yönetim Bütçe Tahmini Görüntüleme Sequence Diyagramı

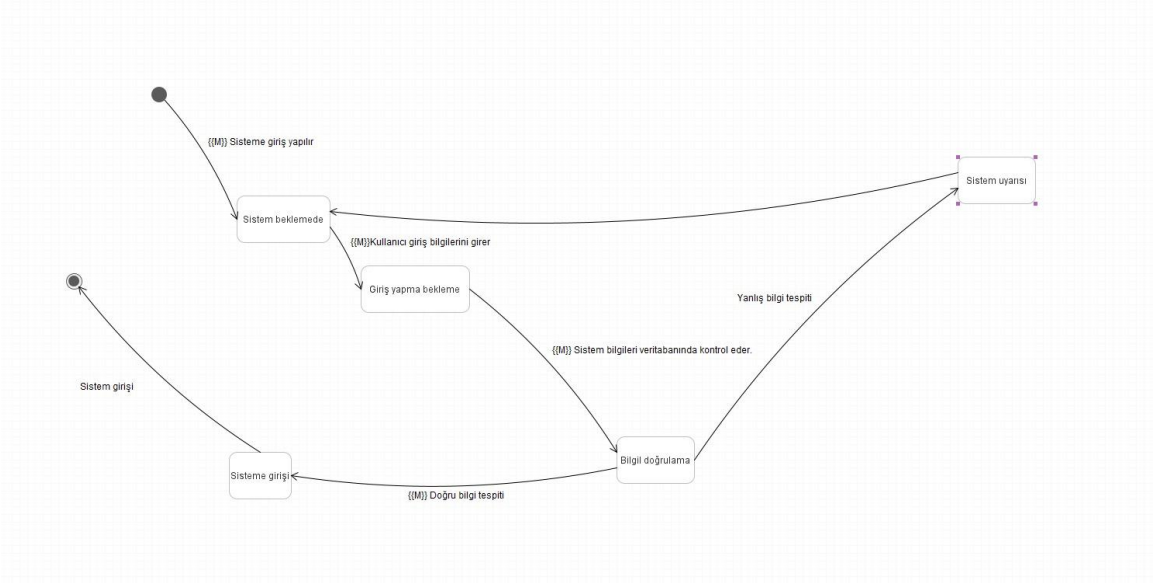


4.1.7 Üst Yönetim Harcama Raporu Görüntüleme Sequence Diyagramı

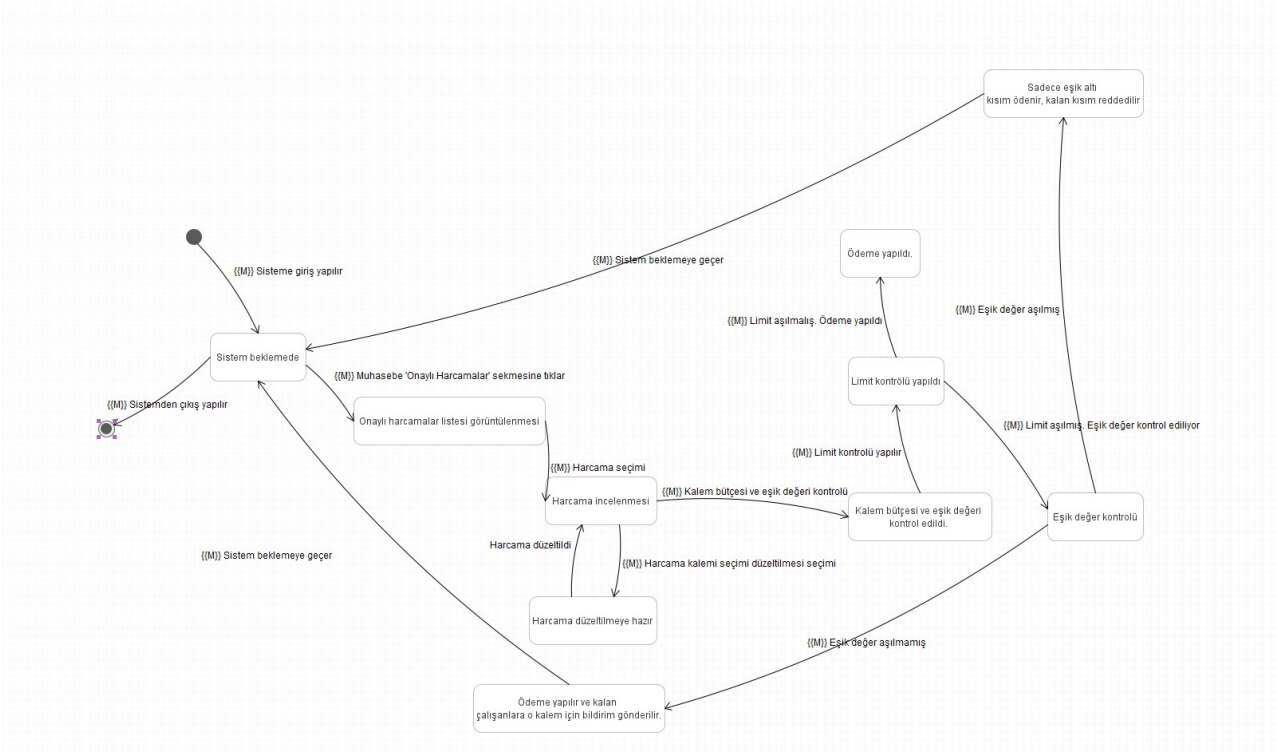


4.2 Durum (State) Diyagramları

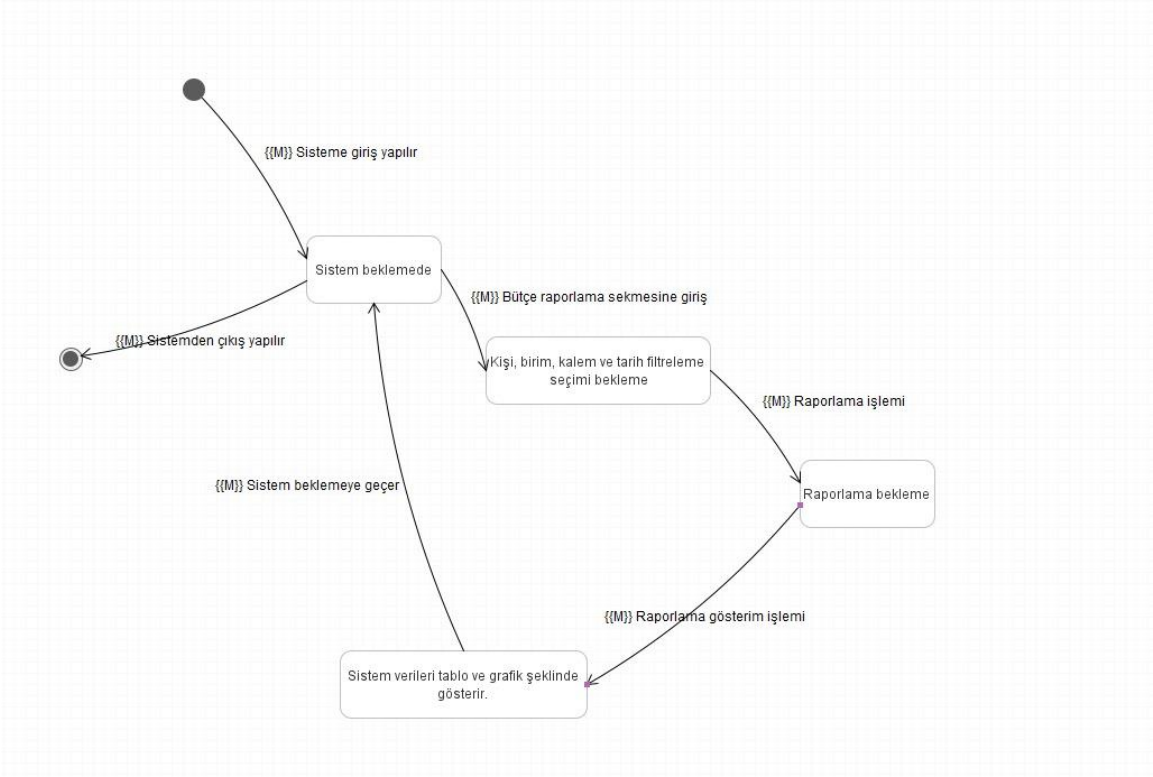
4.2.1 Giriş Yapma Durum Diyagramı



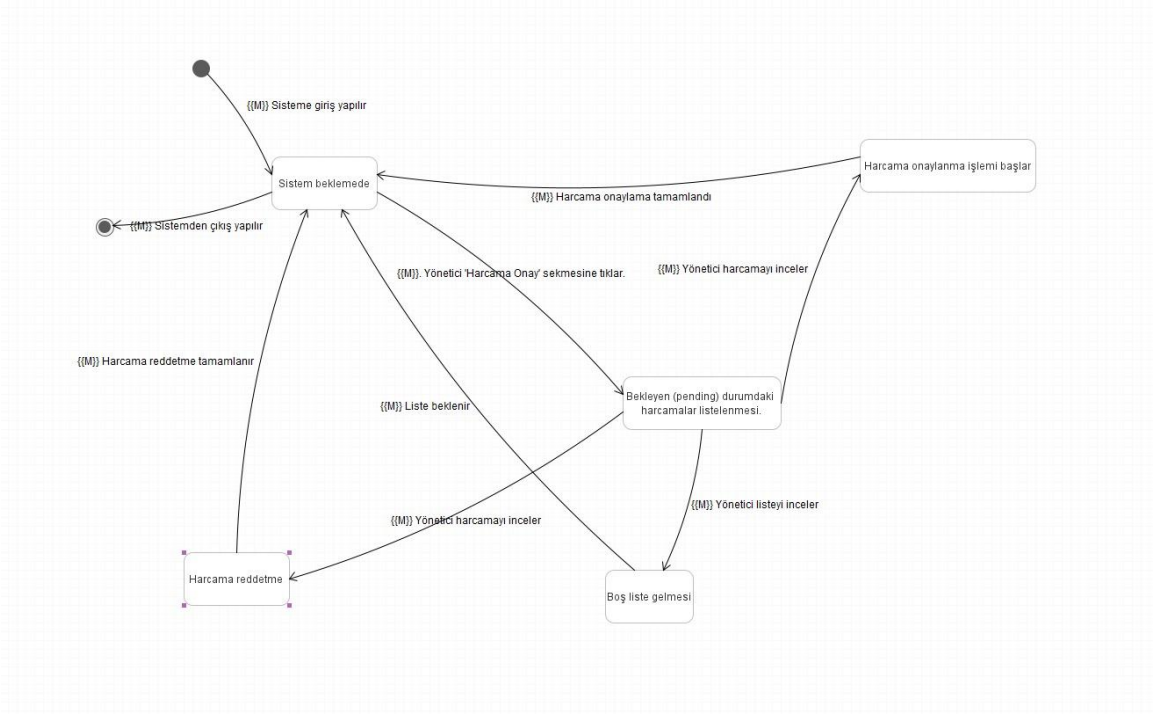
4.2.2 Muhasebe Harcama Tanzim Etme Durum Diyagramı



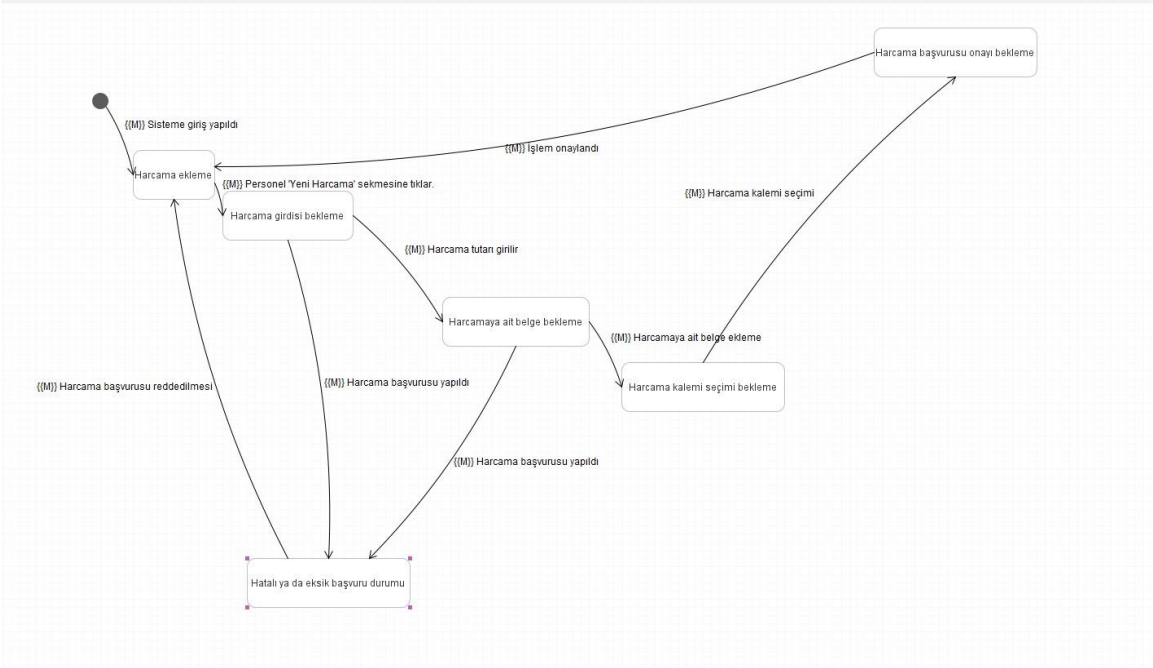
4.2.3 Üst Yönetim Harcama Raporu Görüntüleme Durum Diyagramı



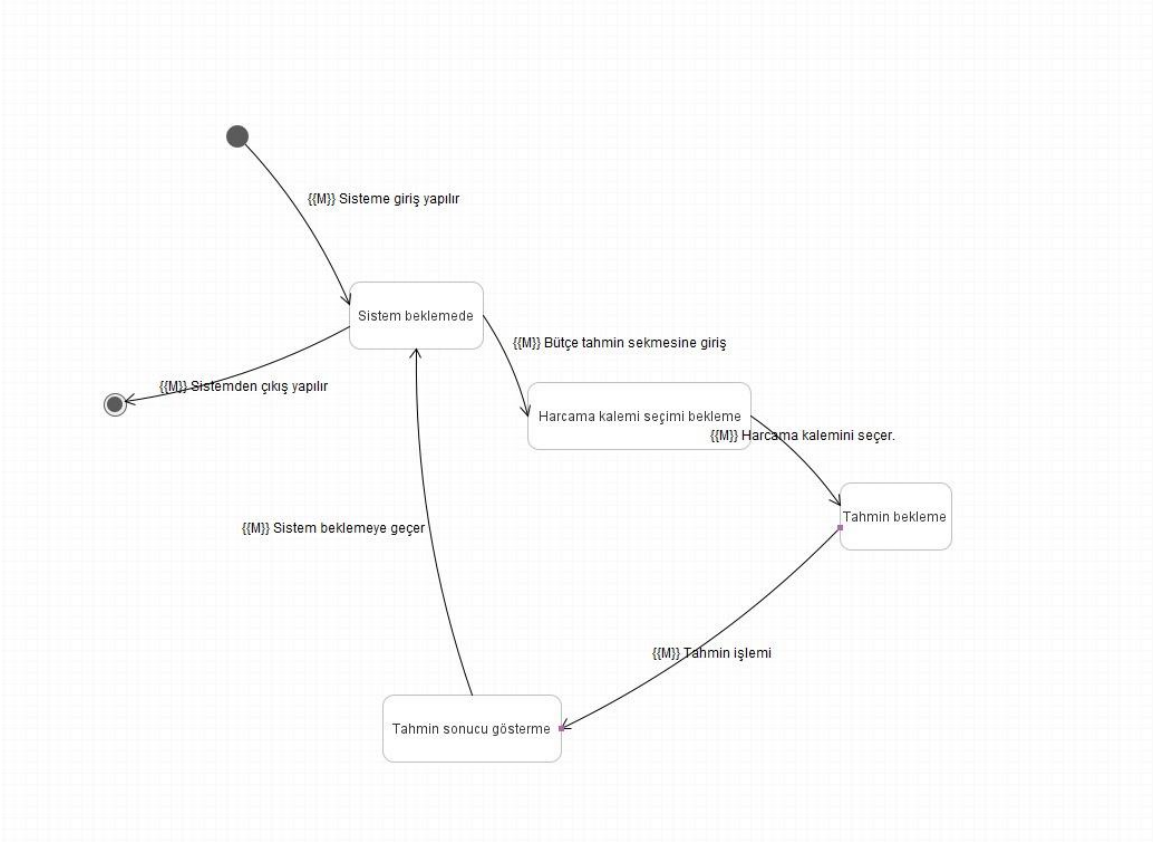
4.2.4 Birim Yöneticisi Harcama Onaylama Durum Diyagramı



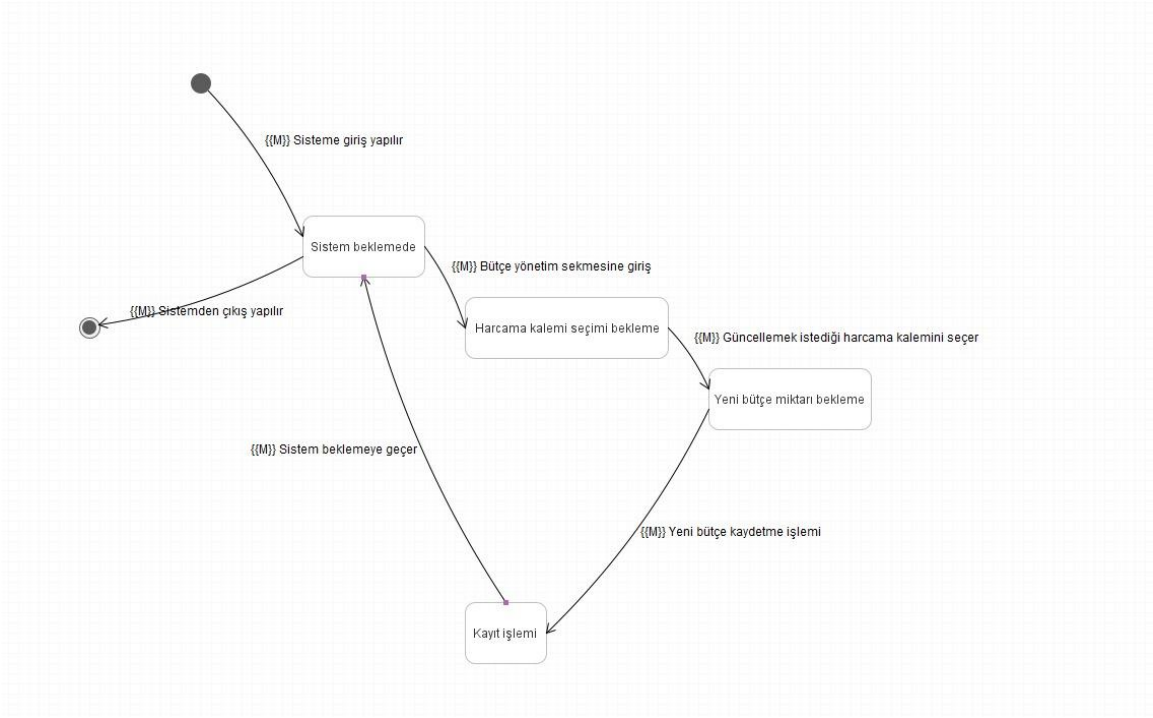
4.2.5 Personel Harcama Ekleme Durum Diyagramı



4.2.6 Üst Yönetim Bütçe Tahmini Görüntüleme Durum Diyagramı

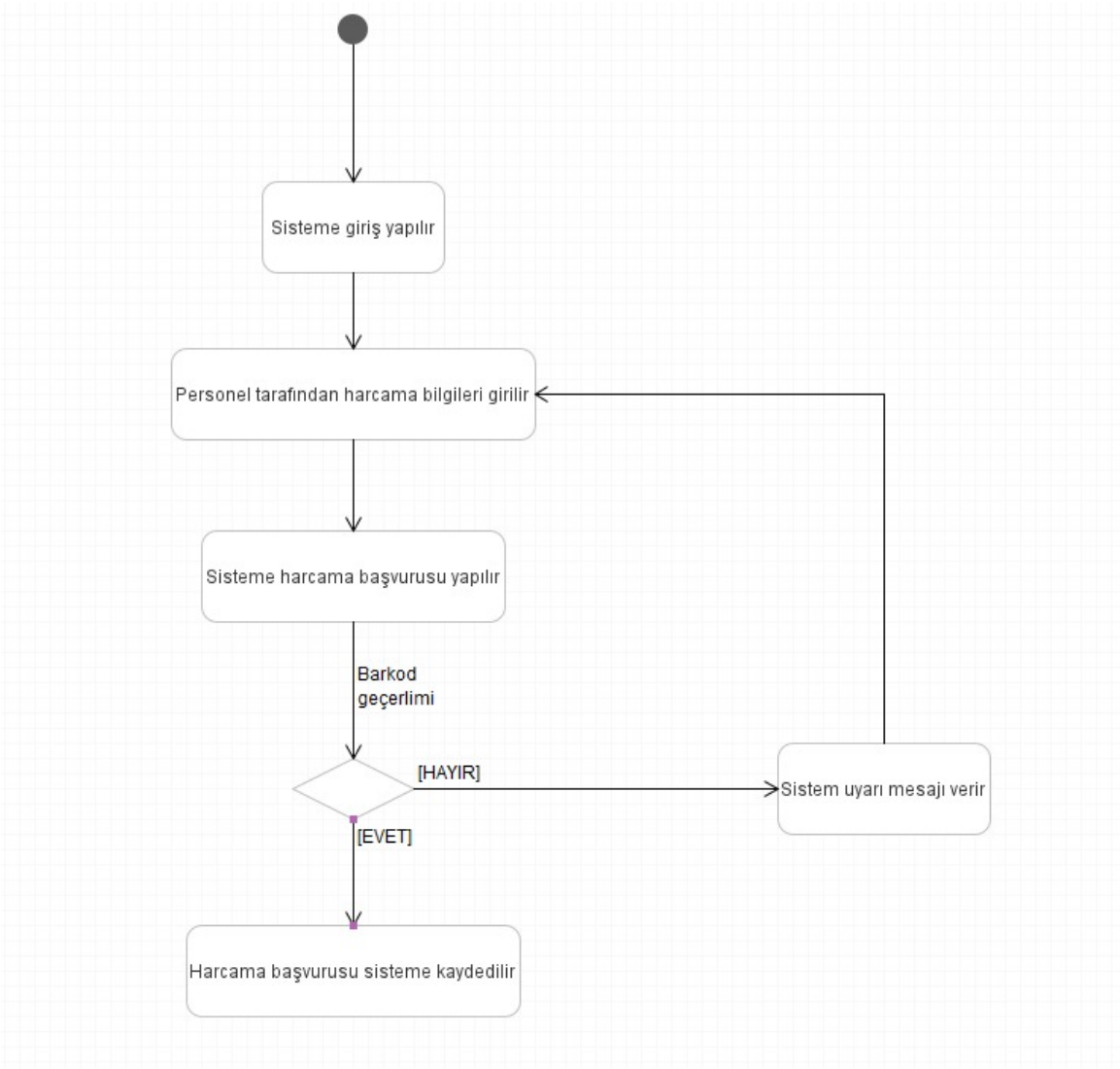


4.2.7 Üst Yönetim Bütçe Güncelleme Durum Diyagramı

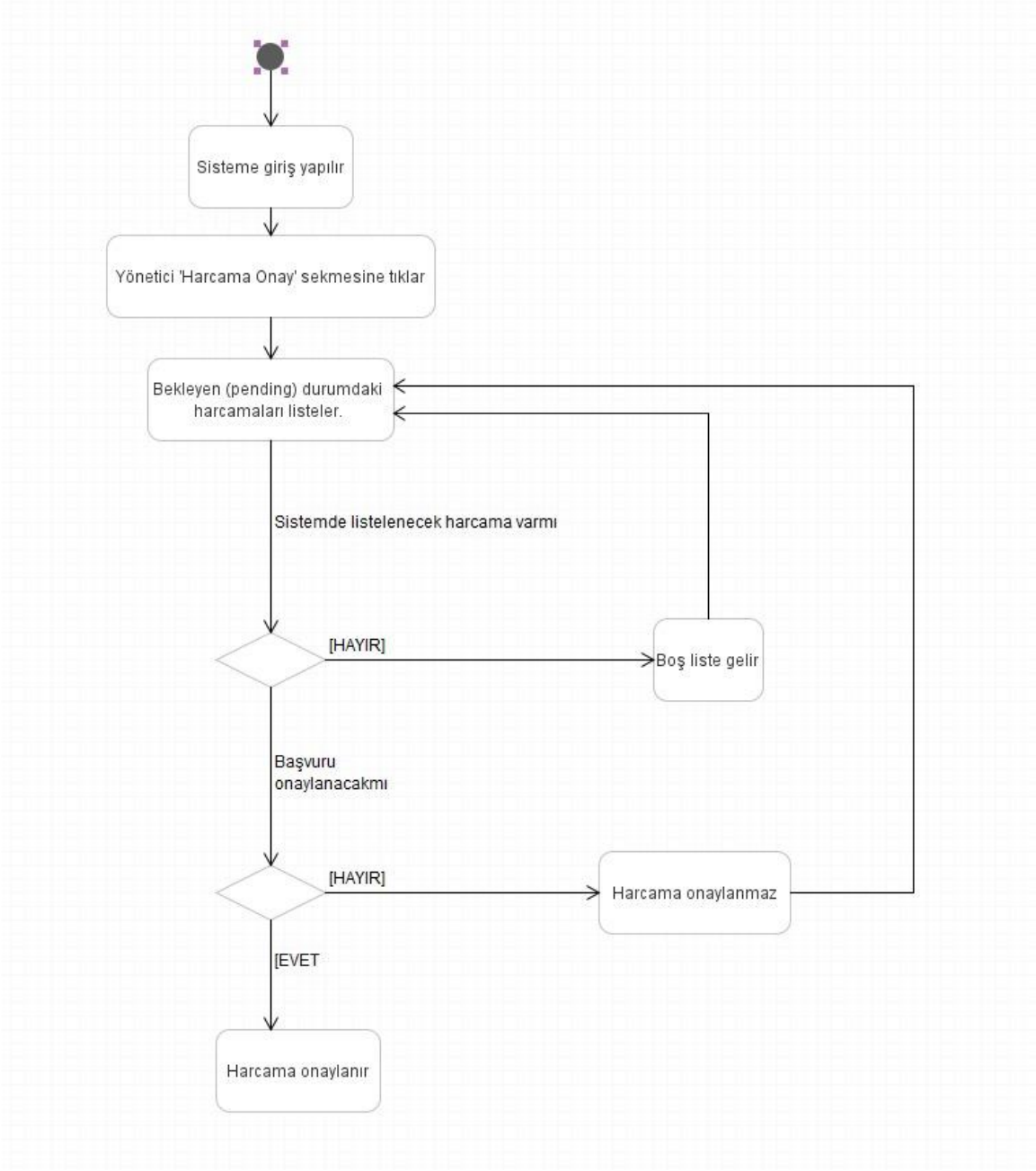


4.3 Etkinlik (Activity) Diyagramları

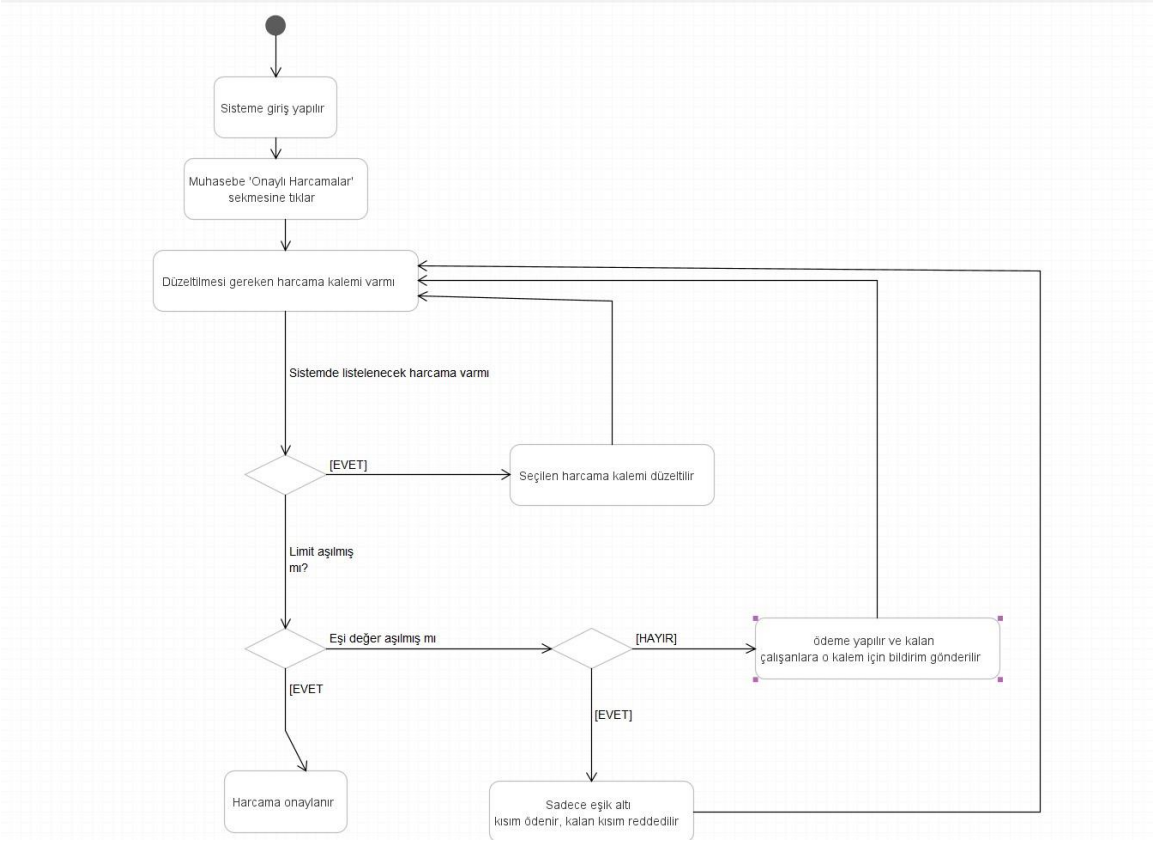
4.3.1 Personel Harcama Ekleme Activity Diyagramı



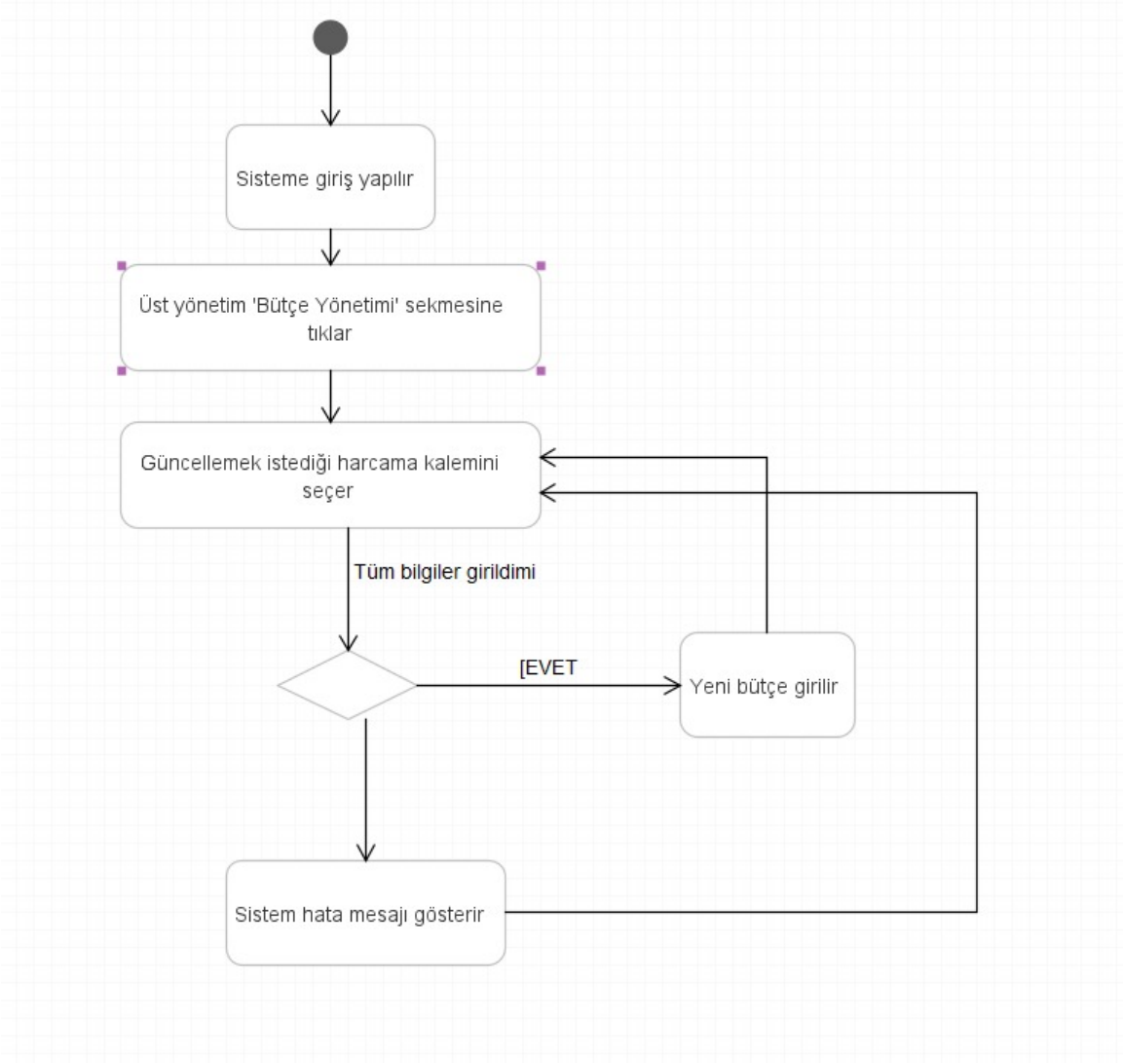
4.3.2 Birim Yöneticisi Harcama Onaylama Activity Diyagramı



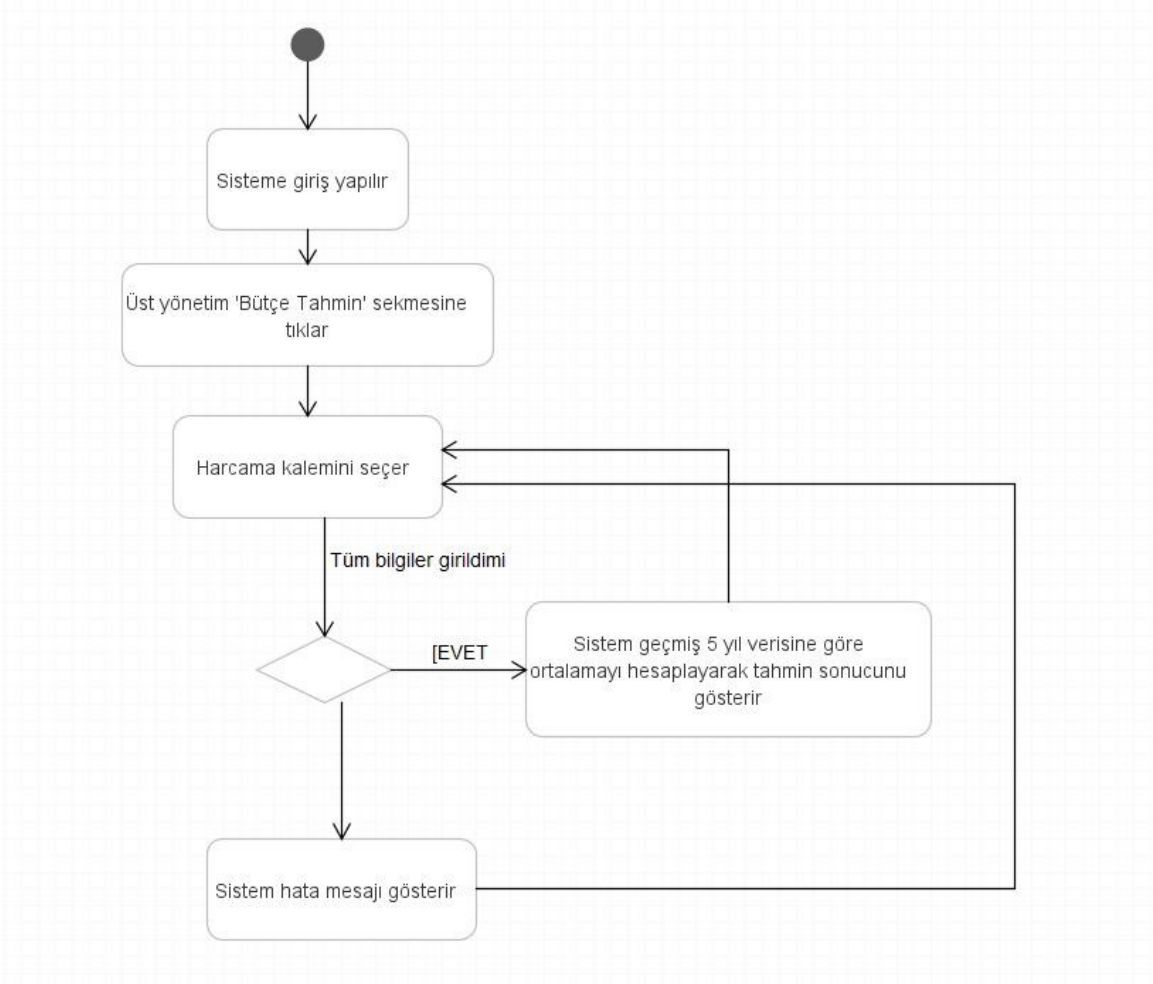
4.3.3 Muhasebe Harcama Tanzim Etme Activity Diyagramı



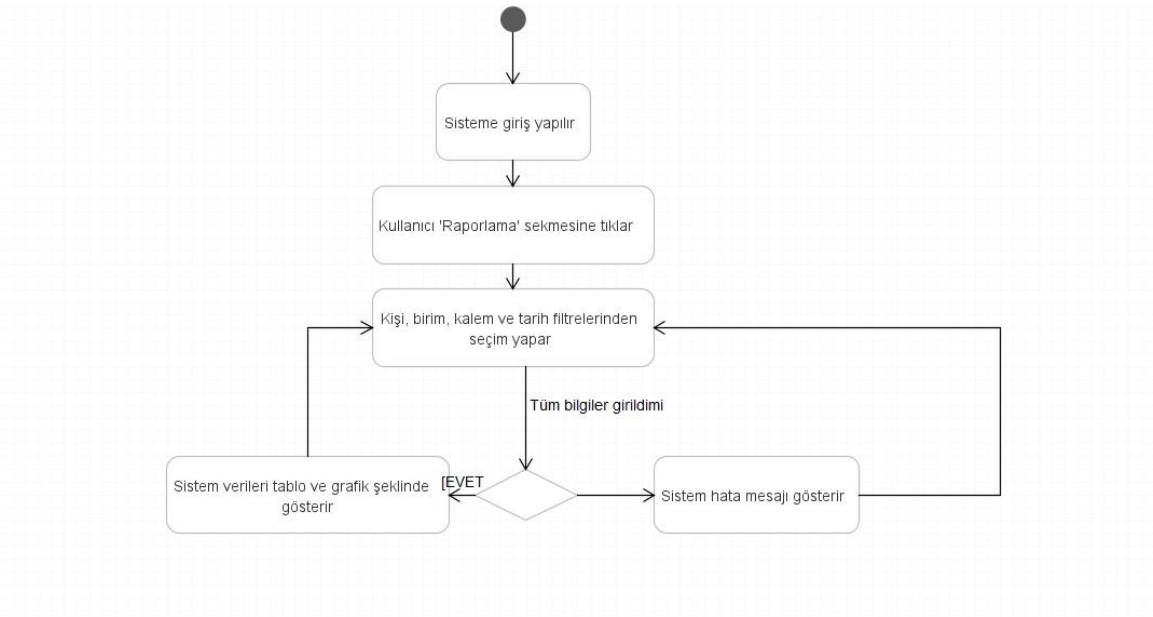
4.3.4 Üst Yönetim Bütçe Güncelleme Activity Diyagramı



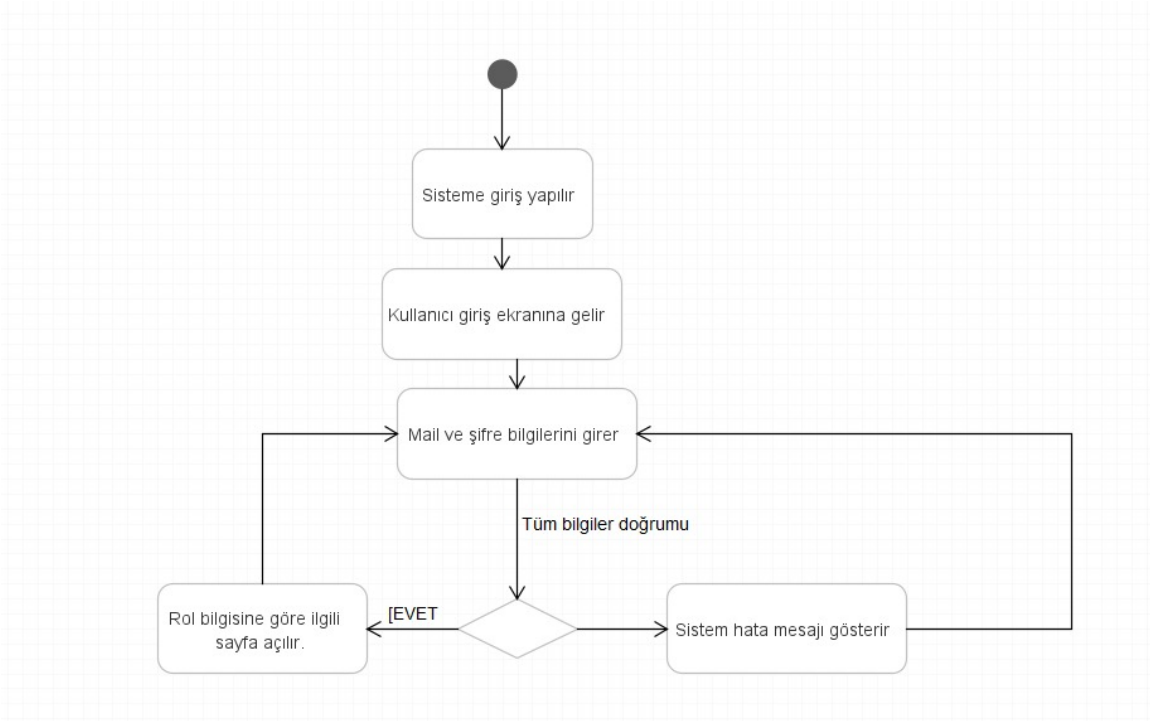
4.3.5 Üst Yönetim Bütçe Tahmini Görüntüleme Activity Diyagramı



4.3.6 Üst Yönetim Harcama Raporu Görüntüleme Activity Diyagramı



4.3.7 Sisteme Giriş Activity Diyagramı



5. TESTLER

ExpenseController.java:

@BeforeEach

```
@BeforeEach
public void setUp() throws SQLException {
    expenseService = mock(IExpenseService.class);
    expenseController = new ExpenseController(expenseService);
    mockConnection = mock(Connection.class);
    mockStatement = mock(PreparedStatement.class);

    // SQL prepare işlemi yapılınca sahte statement dönsün
    when(mockConnection.prepareStatement(anyString())).thenReturn(mockStatement);

    // expenseRepository icinde mockConnection kullanılacak sekilde sahte instance olustur
     expenseRepository = new ExpenseRepositoryImpl(mockConnection); // constructor buna göre olmalı


}
```

Test 1 (Erdem KARACA) :

```java
// ExpenseControllerTest.java
@Test
public void testShowExpenseReport_ReturnsList() {
    List<ExpenseReportDto> mockList = new ArrayList<>();
    when(expenseService.getGroupedExpenseReports()).thenReturn(mockList);

    List<ExpenseReportDto> result = expenseController.showExpenseReports();

    assertNotNull(result);
    assertEquals(0, result.size());
}
```

Test 2 (Erdem KARACA) :

```java
// ExpenseControllerTest.java

@Test
public void testGetPendingExpensesByDepartmentID_ReturnsList() {
    List<Expense> mockList = new ArrayList<>();
    when(expenseService.getPendingExpensesByDepartmentID(5)).thenReturn(mockList);

    List<Expense> result = expenseController.getPendingExpensesByDepartmentID(5);

    assertNotNull(result);
    assertEquals(0, result.size());
    verify(expenseService).getPendingExpensesByDepartmentID(5);
}
```

Test 3 (Ömer Faruk YILDIRIM):

```java
@Test
public void testGetApprovedExpensesByDepartmentID_ReturnsList() {
    List<Expense> mockList = new ArrayList<>();
    when(expenseService.getApprovedExpensesByDepartmentID(3)).thenReturn(mockList);

    List<Expense> result = expenseController.getApprovedExpensesByDepartmentID(3);

    assertNotNull(result);
    assertEquals(0, result.size());
    verify(expenseService).getApprovedExpensesByDepartmentID(3);
}
```

Test 4 (Ömer Faruk YILDIRIM):

```java
@Test
public void testProcessExpensePayment_ReturnsString() {
    String expected = "Success";
    when(expenseService.processExpensePayment(7)).thenReturn(expected);

    String result = expenseController.processExpensePayment(7);

    assertEquals(expected, result);
    verify(expenseService).processExpensePayment(7);
}
```

Test 5 (Cabir COŞKUN):

```java
@Test
public void testCorrectExpenseItem_CallsServiceCorrectly() {
    expenseController.correctExpenseItem(10, 99);
    verify(expenseService).correctExpenseItem(10, 99);
}
```

Test 6 (Cabir COŞKUN):

```java
// ExpenseControllerTest.java
@Test
public void testAddExpense_CreatesAndPassesCorrectExpense() {
    int userId = 1;
    int expenseItemId = 2;
    double amount = 100.0;
    String documentPath = "test.pdf";

    expenseController.addExpense(userId, expenseItemId, amount, documentPath);

    // ArgumentCaptor ile Expense objesinin içeriğini kontrol ediyoruz
    ArgumentCaptor<Expense> expenseCaptor = ArgumentCaptor.forClass(Expense.class);
    verify(expenseService).addExpense(expenseCaptor.capture());

    Expense capturedExpense = expenseCaptor.getValue();
    assertEquals(userId, capturedExpense.getUserId());
    assertEquals(expenseItemId, capturedExpense.getExpenseItemId());
    assertEquals(amount, capturedExpense.getAmount());
    assertEquals(documentPath, capturedExpense.getDocumentPath());
}
```

Test 7 (Tarık TAŞKIN):

```java
@Test
void testRejectExpense_UpdatesStatusToRejected() throws SQLException {
    int testExpenseId = 123;

    // Metodu çağır
    expenseRepository.rejectExpense(testExpenseId);

    // Doğru SQL sorgusu ile hazırlama yapılmış mı?
    verify(mockConnection).prepareStatement("UPDATE expenses SET status = 'REJECTED' WHERE id = ?");

    // Doğru parametre atanmış mı?
    verify(mockStatement).setInt(1, testExpenseId);

    // executeUpdate çağrılmış mı?
    verify(mockStatement).executeUpdate();
}
```

ExpenseItemController.java

@BeforeEach

```java
@BeforeEach
public void setUp() {
    expenseItemService = mock(IExpenseItemService.class);
    expenseItemController = new ExpenseItemController(expenseItemService);
}
```

Test 8 (Tarık TAŞKIN):

```java
// ExpenseItemControllerTest.java
@Test
public void testUpdateExpenseItemBudget_CallsServiceCorrectly() {
    expenseItemController.updateExpenseItemBudget(10, 5000.0);
    verify(expenseItemService).updateExpenseItemBudget(10, 5000.0);
}
```

Test 9 (Yusuf DAMBASAN):

```java
@Test
public void testGetExpenseItems_ReturnsList() {
    List<ExpenseItem> items = new ArrayList<>();
    when(expenseItemService.getExpenseItems()).thenReturn(items);

    List<ExpenseItem> result = expenseItemController.getExpenseItems();

    assertNotNull(result);
    assertEquals(0, result.size());
}
```

UserController.java

@BeforeEach

```
@BeforeEach
public void setUp() {
    userService = mock(IUserService.class);
    userController = new UserController(userService);
}
```

Test 10 (Yusuf DAMBASAN):

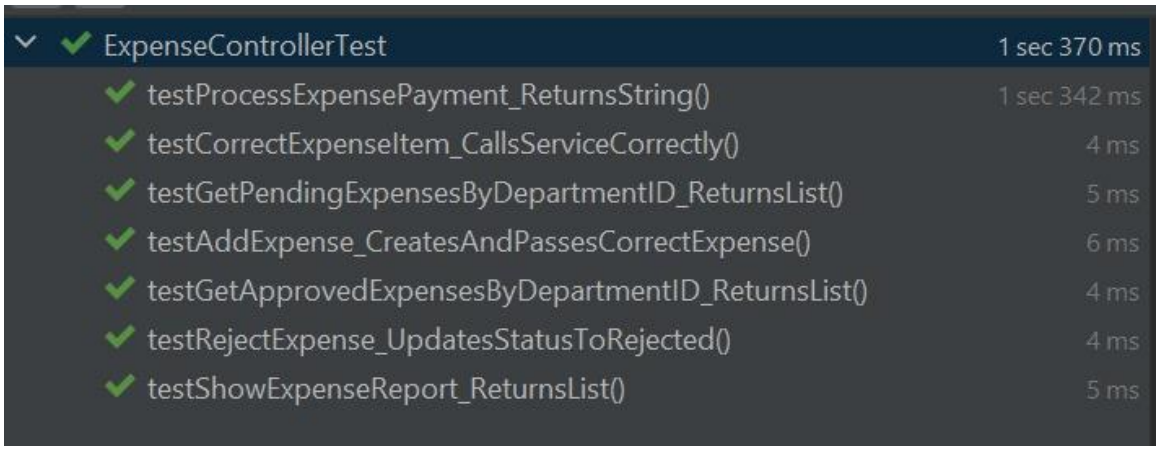
```
// UserControllerTest.java
@Test
public void testLogin_ReturnsUserDto() {
    String email = "test@example.com";
    String password = "pass123";
    UserDto userDto = new UserDto();  // dummy object

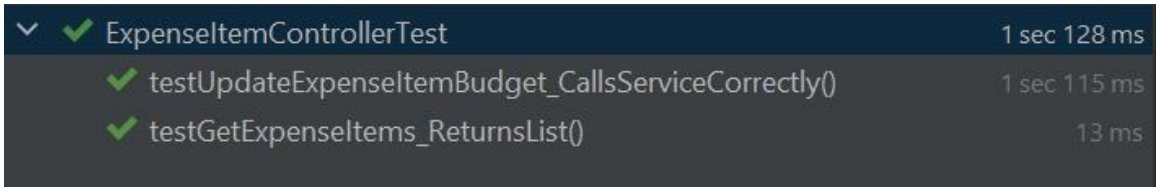
    when(userService.login(any(UserLoginDto.class))).thenReturn(userDto);

    UserDto result = userController.login(email, password);

    assertNotNull(result);
    verify(userService).login(any(UserLoginDto.class));
}
```

Test Sonuçları :






UserControllerTest
1 sec 133 ms
testLogin_ReturnsUserDto()
1 sec 133 ms